LLM FUNDAMENTALS

Complete Beginner-to-Developer Notes

Learn how Large Language Models work, how to use their APIs, and how to build practical AI applications with Python and FastAPI.

Includes: AI foundations • tokens • embeddings • Transformers • attention • training • inference • prompting • APIs • RAG • fine-tuning • safety • practical project

Prepared as a study guide | November 2026

How to use these notes

Study the concepts in order. First understand the mental model of an LLM, then learn the architecture, and finally practise calling a model through an API. You do not need to implement a Transformer from scratch to become an AI application developer.

Learning outcomes

• Explain what an LLM is and how next-token prediction works.

• Understand tokens, token IDs, embeddings, context windows and attention.

• Distinguish pretraining, fine-tuning, inference and retrieval-augmented generation.

• Use an LLM API from Python and connect it to a FastAPI backend.

• Recognise hallucinations, privacy risks, prompt injection and evaluation needs.

Contents

Section	Topics
1	AI, ML, Deep Learning, NLP and LLM
2	What an LLM is and how text generation works
3	Tokens, tokenization and context windows
4	Embeddings, vectors and semantic similarity
5	Transformer architecture and attention
6	Training, fine-tuning and inference
7	Model families, APIs and generation settings
8	Prompt engineering, hallucinations and safety
9	RAG, tools and AI application architecture
10	Python + FastAPI example, project and glossary

1. AI, ML, Deep Learning, NLP and LLM

Artificial Intelligence (AI) is the broad field of building computer systems that perform tasks associated with intelligent behaviour, such as perception, language, planning or decision support.

Machine Learning (ML) is a branch of AI in which systems learn patterns from data instead of relying only on manually written rules.

Deep Learning (DL) is a branch of ML that uses neural networks with multiple layers to learn complex patterns.

Natural Language Processing (NLP) is the area of AI concerned with processing and generating human language: classification, translation, summarisation, search and more.

Large Language Model (LLM) is a neural network trained on large datasets to process and generate language. Many modern LLMs use the Transformer architecture and are trained to predict the next token.

Term	Simple meaning	Example
AI	Broad goal: intelligent computer behaviour	An assistant that answers questions
ML	Learn patterns from examples	Classify support tickets

Term	Simple meaning	Example
Deep Learning	Learn using multi-layer neural networks	Recognise speech or images
NLP	Work with human language	Translate English to Telugu
LLM	Large language model for language tasks	Draft an email or explain Python

Relationship: AI is the broad field; ML is one approach within AI; deep learning is one approach within ML; NLP is a language-focused field that can use ML and deep learning; LLMs are one family of deep-learning models used for language.

2. What is an LLM?

An LLM is a neural network with learned numerical parameters (often called weights). During training, it learns statistical patterns in text and other training data. A common training objective is to predict the next token from the tokens that came before it.

Next-token prediction

Imagine the input is: **“The capital of France is”**. The model calculates scores or probabilities for possible next tokens. It may assign a high probability to “Paris”. After selecting a token, it appends it to the sequence and predicts again.

```
Input tokens:  The | capital | of | France | is
                          |
                          v
                 Model predicts next token
                          |
                          v
                        Paris
                          |
                          v
              Predict the next token again
```

The model repeats this process until it reaches a stop condition or output limit. This is called **autoregressive generation**.

What an LLM does and does not do

• It can generate text, explain concepts, summarise, translate, classify, draft code and produce structured outputs.

• It does not automatically know live information. Current data may require search, tools or a connected data source.

• It does not automatically execute Python, SQL, or external actions. Your application must execute approved tool calls.

• A fluent answer is not proof that the answer is true. Verify important outputs.

Important mental model: an LLM predicts and generates tokens. The surrounding application supplies data, validates results, executes tools and enforces permissions.

3. Tokens, tokenization and context windows

What is a token?

A token is a unit of text processed by a model. It may be a whole word, part of a word, punctuation, whitespace or another character sequence. Tokenization is the process that converts text into token units.

```
Example only (actual splits depend on the tokenizer):
"unbelievable!" -> ["un", "believ", "able", "!"]

Text -> Tokenizer -> Token IDs -> LLM
LLM output token IDs -> Decoder -> Text
```

Each token maps to an integer ID from a tokenizer vocabulary. The model processes numerical representations, not ordinary words directly.

Why tokens matter

• **Cost:** many API providers calculate usage from input and output tokens.

• **Context:** model input limits are measured in tokens, not pages or words.

• **Latency:** longer inputs and outputs can take more time to process.

• **Language:** the same sentence can have different token counts across languages and tokenizers.

A rough English estimate is about 1 token per 4 characters or 0.75 words. This is only an estimate; never use it as an exact count. Use the tokenizer for the specific model when precision matters.

Input and output tokens

Type	Meaning
Input tokens	Instructions, user message, conversation history and supplied context sent to the model.
Output tokens	Tokens generated by the model.
Cached input tokens	Previously processed input that may be reused by supported APIs.
Reasoning tokens	Some reasoning models use additional internal tokens; API usage rules vary by model.

Context window

The context window is the maximum amount of token information a model can process for a request or interaction, subject to the model's specific limits. It may include system instructions, tool definitions, chat history, user input, retrieved documents and generated output.

Example: if a model supports a 32,000-token context, the combined prompt and generated tokens must fit within the applicable limits. The exact input/output rules differ by model and API.

A context window is not the same as permanent memory. If an application needs information from previous sessions, it must store and retrieve relevant information.

4. Embeddings, vectors and semantic similarity

An embedding is a list of numbers (a vector) that represents information such as text. An embedding model maps text into a numerical space where semantically related pieces of text often have nearby representations.

```
Text A: "How do I reset my password?"
       -> [0.12, -0.44, 0.81, ...]

Text B: "I forgot my login password."
       -> [0.10, -0.39, 0.78, ...]

The vectors may be close because the meanings are similar.
```

Embeddings are useful for semantic search, document retrieval, clustering, recommendations and duplicate detection. They are not the same thing as the LLM's generated answer.

Keyword search vs semantic search

Keyword search	Semantic search
Looks for matching words or terms.	Looks for similarity in meaning.
May miss a paraphrase with different words.	Can find related wording even when exact terms differ.

Keyword search	Semantic search
Useful for exact names, IDs and phrases.	Useful for natural-language questions and document discovery.

For a document Q&A; system, split documents into chunks, create an embedding for each chunk, store vectors in a vector database or search index, retrieve relevant chunks for a question, and pass those chunks to the LLM.

5. Transformer architecture and attention

The Transformer is a neural-network architecture introduced for sequence processing. It is the foundation of many widely used LLMs. A typical text-generation path includes tokenization, token embeddings, positional information, repeated Transformer blocks and an output layer.

```
Text
  |
Tokenizer -> Token IDs
  |
Embedding + position information
  |
Transformer blocks (attention + feed-forward layers)
  |
Output scores for next token
  |
Select token -> Decode to text
```

Embeddings and position

Token embeddings turn token IDs into vectors. Position information helps the model represent token order. Without order information, “dog bites person” and “person bites dog” could be harder to distinguish.

Self-attention

Self-attention lets each token representation incorporate information from other tokens in the sequence. It helps the model use context to interpret a word or phrase.

A common conceptual description uses three learned projections:

• **Query (Q):** what information this token is looking for.

• **Key (K):** what information each token makes available for matching.

• **Value (V):** the information that can be combined into the updated representation.

The scaled dot-product attention operation is commonly written as:

Attention(Q, K, V) = softmax(QK■ / √d■) V

You do not need to memorise the formula first. Understand the intuition: compare queries with keys, turn scores into weights, then combine values.

Multi-head attention and feed-forward layers

Multi-head attention runs several attention operations in parallel so the model can learn different relationships. Feed-forward layers then transform each token representation. Transformer blocks also include residual connections and normalisation in common designs.

Encoder, decoder and decoder-only models

• **Encoder:** builds contextual representations of input; commonly useful for understanding tasks.

• **Decoder:** generates a sequence token by token.

• **Encoder-decoder:** uses an encoder to represent input and a decoder to generate output; common in translation-style tasks.

• **Decoder-only:** predicts next tokens and is widely used for generative chat LLMs.

6. Training, fine-tuning and inference

Pretraining

Pretraining exposes a model to large datasets and teaches it broad patterns by optimising a training objective, commonly next-token prediction for autoregressive models. The result is a base model.

Post-training and instruction tuning

Post-training adapts a base model to behave more usefully. It can include supervised fine-tuning on curated examples, preference optimisation or reinforcement-learning methods, safety training and other techniques. The exact recipe varies by model.

Fine-tuning

Fine-tuning continues training a pretrained model on a more targeted dataset. It can help adapt style, task behaviour or domain patterns. It is not automatically the best way to add frequently changing facts; retrieval is often better for that.

Inference

Inference is using a trained model to produce an output. A simplified flow:

```
1. Application builds the request.
2. Text/messages are tokenized.
3. Model processes the input.
4. Model calculates next-token scores.
5. Decoding selects a token.
6. Repeat until stop condition or output limit.
7. Application receives and displays the response.
```

Training changes model parameters. **Inference** uses the trained parameters to produce an output.

7. Model families, APIs and generation settings

Common model categories

Category	Typical use
General-purpose LLM	Chat, writing, summarisation, coding and reasoning.
Reasoning model	Complex multi-step problems, logic, maths and coding.
Embedding model	Semantic search, clustering and retrieval.
Multimodal model	Works with combinations of text, images, audio or video, depending on capability.
Speech model	Speech recognition, transcription or speech generation.

Examples of model providers and ecosystems include OpenAI, Google, Anthropic, Meta's Llama ecosystem, Mistral and models distributed through Hugging Face. Model availability, names, capabilities and prices change; check the provider's current model documentation before implementation.

What is an LLM API?

An API (Application Programming Interface) lets your application send a request to a model provider and receive a response. Your Python backend can call the provider without exposing the secret API key to the browser.

```
Frontend (React / Next.js)
       |
       | HTTPS request to your backend
```

```
       v
Python backend (FastAPI)
       |
       | API key stored securely on server
       v
LLM provider API
       |
       v
Model response -> FastAPI -> Frontend
```

Generation settings

Setting	Meaning
Temperature	Controls randomness in token selection. Lower values are generally more focused; higher values can be more vari
Top-p	Limits selection to a probability mass of likely next tokens (nucleus sampling).
Maximum output tokens	Caps the generated output length, subject to API/model rules.
Stop conditions	Tell generation when to stop, where supported.
Streaming	Sends partial output as it is generated instead of waiting for the complete response.

Not every model or endpoint supports every parameter. Always follow the API documentation for the exact model and endpoint.

8. Prompt engineering, hallucinations and safety

Prompt structure

A prompt is the input that guides a model. Clear prompts state the task, relevant context, constraints and desired output format.

```
Role: You are a helpful HRMS support assistant.
Task: Explain a timesheet validation error.
Context: The employee's work order starts on 10 November.
Constraint: Do not invent missing dates.
Output: Give the likely cause and a short troubleshooting checklist.
```

Practical prompting techniques

• Be specific about the task and intended audience.

• Provide relevant context, examples and definitions.

• Specify output format: bullets, JSON, table or short explanation.

• Break complex tasks into smaller steps.

• Ask the model to state uncertainty and identify missing information.

• Validate structured output in your application; never trust model-generated JSON without parsing and schema validation.

Hallucination

A hallucination is an output that sounds plausible but is false, unsupported or fabricated. LLMs can produce hallucinations because generation is based on learned patterns and the information in the current context, not a guaranteed fact-checking mechanism.

Reduce risk: provide trusted source material, use retrieval or tools for current facts, require citations where appropriate, validate calculations and IDs in code, and route high-impact decisions to a qualified human.

Security and privacy

• Never put secret API keys in frontend JavaScript, public repositories or screenshots.

• Keep credentials in environment variables or a secrets manager; rotate a key if exposed.

• Send only the personal or business data needed for the task.

• Treat user documents and retrieved text as untrusted input.

• Defend against prompt injection: document text must not override system policy or authorise actions.

• Use authentication, authorisation, rate limits, logging controls and human approval for consequential actions.

9. RAG, tools and AI application architecture

What is RAG?

Retrieval-Augmented Generation (RAG) combines information retrieval with LLM generation. Instead of relying only on the model's pretrained knowledge, the application retrieves relevant content from a trusted data source and includes it in the prompt.

```
Documents
   |
Clean and split into chunks
   |
Create embeddings
   |
Store in vector search index
   |
User asks a question
   |
Embed question and retrieve relevant chunks
   |
Build prompt with retrieved context
   |
LLM generates grounded answer
   |
Return answer (ideally with source references)
```

RAG vs fine-tuning

RAG	Fine-tuning
Adds relevant external information at request time.	Changes model behaviour by further training.
Good for private, changing or source-based knowledge.	Good for consistent task patterns, style or specialised behaviour.
Documents can be updated without retraining the LLM.	Requires a training workflow and evaluation.
Can provide source references from retrieved material.	Does not inherently provide citations to source documents.

Tool calling

Tool calling lets a model request an action in a structured format, such as looking up an employee record or checking an order. The application validates the request, checks permissions, executes the actual function, and returns the result to the model. The model itself does not gain direct database access merely because a tool is described.

Example HRMS use cases

• Ask questions about an employee handbook using RAG.

• Summarise a timesheet exception using approved timesheet data.

• Draft onboarding emails from validated employee and joining information.

• Classify support tickets and suggest a response.

• Search policy documents and show the source section.

Critical design rule: an LLM can suggest an action, but normal backend code must enforce business rules, permissions, data validation and audit logging.

10. Python + FastAPI: a minimal API integration pattern

The example below demonstrates the architecture. It uses a placeholder for the model call: follow the provider's current SDK and API documentation for the exact method, model name and response fields. Keep your API key on the backend.

Install packages

```
python -m venv venv
# Windows PowerShell:
.\venv\Scripts\Activate.ps1

pip install fastapi uvicorn openai python-dotenv
```

Environment file

```
# .env (do not commit this file)
OPENAI_API_KEY=your_real_secret_api_key
```

FastAPI backend example

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from dotenv import load_dotenv
from openai import OpenAI
import os

load_dotenv()

app = FastAPI()
api_key = os.getenv("OPENAI_API_KEY")
if not api_key:
    raise RuntimeError("OPENAI_API_KEY is missing")

client = OpenAI(api_key=api_key)

class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
def chat(request: ChatRequest):
    try:
        response = client.responses.create(
            model="MODEL_NAME_SUPPORTED_BY_YOUR_ACCOUNT",
            input=request.message
        )
        return {"answer": response.output_text}
    except Exception:
        # Log safe diagnostic details on the server.
        raise HTTPException(
            status_code=502,
            detail="The AI service request failed"
        )
```

Replace the model placeholder with a model currently available to your API project. A tracking ID, project ID or organisation ID is not a secret API key. The API key is a credential used to authenticate requests. Never share it in chat or commit it to Git.

Typical debugging checklist

• 401 Unauthorized: verify that the correct secret key is loaded, active and belongs to the intended project/organisation.

• 403 Forbidden: check project permissions, model access and policy restrictions.

• 404 Not Found: check the route, endpoint and model identifier.

• 429 Too Many Requests: check rate limits, quota and billing configuration.

• 500/502/503: inspect provider status and server-side error logs; retry safely when appropriate.

• FastAPI 404: confirm the URL path and HTTP method match the route you defined.

11. Practical mini-project: HRMS policy assistant

Goal: build a chatbot that answers employee questions using HRMS policy documents.

Suggested build stages

Stage	Build task	What you learn
1	Create FastAPI /chat endpoint	HTTP, request/response, Pydantic
2	Connect an LLM API	API key, SDK, inference
3	Store chat sessions in MySQL	Database design and history
4	Upload and parse policy documents	File handling and text extraction
5	Chunk text and create embeddings	Embeddings and retrieval
6	Retrieve relevant chunks and call the LLM	RAG
7	Show sources and handle unknown answers	Grounding and evaluation
8	Add login, roles, logs and rate limits	Production security

Sample test questions

• How many days of annual leave are available?

• What is the process for requesting leave?

• Which document is required during onboarding?

• If the policy document does not contain the answer, does the assistant admit it?

Evaluation checklist

• Is the answer supported by the retrieved policy text?

• Does it cite or identify the relevant document section?

• Does it say “I could not find this in the provided policy” when evidence is missing?

• Does it prevent one employee from seeing another employee's private information?

• Are API errors handled without exposing keys or internal stack traces?

12. Quick revision and glossary

Term	Meaning
LLM	Large Language Model; a neural network trained to process and generate language.
Token	A unit of text processed by a model.
Tokenizer	Converts text into token IDs and decodes token IDs back into text.
Embedding	A numerical vector representation of data.

Term	Meaning
Vector database	A system that stores and searches vector representations.
Transformer	Neural-network architecture built around attention mechanisms.
Attention	Mechanism that combines information across tokens based on learned relevance.
Pretraining	Initial large-scale training of a model.
Fine-tuning	Further training to adapt a pretrained model.
Inference	Using a trained model to generate an output.
Context window	The token capacity available to a model for a request.
Prompt	Instructions and context supplied to a model.
Hallucination	Plausible-sounding but false or unsupported output.
RAG	Retrieving external information and supplying it to the LLM.
API	A defined interface for software systems to communicate.
Streaming	Receiving generated output progressively.
Tool calling	A model requests a structured tool action that application code executes.

Self-assessment questions

• What is the difference between AI, ML, deep learning and an LLM?

• Why does an LLM use tokens instead of processing a sentence as plain words?

• What is the difference between an embedding and a token ID?

• What problem does self-attention help solve?

• How are training and inference different?

• What is a context window?

• When would you choose RAG instead of fine-tuning?

• Why should an API key stay on the backend?

• Does an LLM execute a database query by itself?

• How would you test whether an HRMS assistant gives grounded answers?

Suggested 7-day study plan

Day	Study focus	Practice
1	AI, ML, NLP and LLM basics	Explain the difference in your own words.
2	Tokens and context windows	Compare token counts for different prompts.
3	Embeddings and semantic search	Compare two similar HR policy questions.
4	Transformers and attention	Draw the text-to-token-to-output flow.
5	Training, inference and prompting	Write prompts for three HRMS tasks.
6	APIs and FastAPI	Build and test a /chat endpoint.
7	RAG and evaluation	Design a policy Q&A test set.

References and further learning

Use official documentation for implementation details because model names, supported parameters, SDK methods, token limits and prices can change.

- **Microsoft Learn — LLM fundamentals**
https://learn.microsoft.com/en-us/agent-framework/journey/llm-fundamentals
- **OpenAI API — Concepts**
https://developers.openai.com/api/docs/concepts
- **OpenAI Help — Understanding and counting tokens**
https://help.openai.com/en/articles/4936856-understanding-and-counting-tokens
- **DeepLearning.AI — How Transformer LLMs Work**
https://www.deeplearning.ai/courses/how-transformer-llms-work

These notes are educational material. Always check the current provider documentation and your organisation's security and privacy requirements before deploying an AI application.